

GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India



Foodie-Hub

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

Aarsh Singh

Roll No.: 22515000003 | Section: NB

Under the Supervision of

Gautam Mukherjee

GLA University, Greater Noida

Reviewed By

Gautam Mukherjee

GLA University, Greater Noida

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS

GLA UNIVERSITY, GREATER NOIDA – 201310

October-2026

Revision History

Version	Date	Description	Author
1.0	—	Initial SRS draft	Project Team
2.0	10-10-2026	Expanded edition — added detailed use cases, data dictionary, NFR detail, risk analysis, testing plan, glossary, wireframe notes, and traceability matrix	Project Team

Table of Contents

1. Introduction.....	3
2. Overall Description.....	5
3. Functional Requirements.....	7
4. Shopping Cart Requirements.....	9
5. Checkout Requirements.....	10
6. Order Requirements.....	11
7. Non-Functional Requirements.....	12
8. Hardware Requirements.....	14
9. Software Requirements.....	14
10. System Constraints.....	15
11. Data Requirements.....	16
12. External Interface Requirements.....	18
13. Use Case Specifications.....	19
14. Data Flow Diagram (DFD) Relationship.....	24
15. Requirements Traceability Matrix (Sample).....	25
16. Testing Plan.....	26
17. Risk Analysis.....	27
18. Future Enhancements.....	28
19. Acceptance Criteria.....	29
20. Glossary.....	30
21. Conclusion.....	31

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe, in detail, the functional and non-functional requirements of the FoodieHub – Food Delivery Web Application. FoodieHub is a responsive web-based application designed to provide users with a simple and convenient platform for browsing restaurants, viewing food menus, searching and filtering food items, managing a shopping cart, and simulating the food-ordering process.

This document is intended for developers, testers, project supervisors/evaluators, and any stakeholder who needs a complete and unambiguous understanding of what the system will do, how it will behave, and the constraints under which it operates. This expanded edition adds greater specification depth — detailed use cases, a data dictionary, a requirements traceability matrix, a risk register, and a testing plan — so that the document can serve as a standalone reference throughout design, implementation, and evaluation.

1.2 Scope of the Project

FoodieHub provides an interactive interface through which users can browse restaurants, view restaurant information and menus, search and filter food items, manage a shopping cart, enter delivery information, select a payment method, and simulate placing an order.

The current version is a front-end application. Real-time delivery tracking, real payment processing, user authentication, and a centralized backend database are outside the scope of the current implementation. These are documented as Future Enhancements (Section 16).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
DOM	Document Object Model — the browser's in-memory representation of a web page
DFD	Data Flow Diagram
UI / UX	User Interface / User Experience
Local Storage	A browser API that persists key-value data on the client with no expiration
Cart	The temporary collection of food items a user intends to order
Checkout	The process of reviewing the cart, entering delivery details, and confirming an order
Responsive Design	A design approach ensuring a UI adapts correctly across screen sizes
CRUD	Create, Read, Update, Delete — basic data operations
WCAG	Web Content Accessibility Guidelines

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- HTML5 Specification — W3C/WHATWG

- [Bootstrap 5 Documentation](#)
- [MDN Web Docs — Web Storage API \(Local Storage\)](#)
- [Web Content Accessibility Guidelines \(WCAG\) 2.1](#)

1.5 Document Overview

Section 2 describes the overall product context. Section 3 details functional requirements by module. Sections 4–6 expand the Cart, Checkout, and Order flows. Section 7 covers non-functional requirements. Sections 8–9 list hardware/software needs. Section 10 lists system constraints. Section 11 provides the data dictionary. Section 12 details external interfaces. Section 13 provides detailed use cases. Section 14 covers DFDs. Section 15 is the requirements traceability matrix. Section 16 is the test plan. Section 17 is the risk register. Section 18 lists future enhancements. Section 19 lists acceptance criteria. Section 20 is a glossary, and Section 21 concludes the document.

2. Overall Description

2.1 Product Perspective

FoodieHub is designed as a standalone front-end web application. It can run directly in a modern web browser without requiring a dedicated server or backend system. The application uses predefined or dummy restaurant and food data. Local Storage can be used to preserve cart and order information within the user's browser.

Basic System Flow: User → Browse Restaurants → View Menu → Search/Filter → Add to Cart → Checkout → Confirm Order

2.1.1 System Context

The application operates entirely client-side. All data — restaurant listings, menus, cart contents, and orders — lives in the browser's memory and/or Local Storage for the duration of a session (or longer, where Local Storage is used). No requests are made to an external application server; the only network dependency is the initial download of the page's HTML, CSS, JavaScript, and image assets.

2.2 Product Functions

- User navigation
- Restaurant browsing
- Menu viewing
- Food searching
- Food filtering
- Shopping cart management
- Price calculation
- Checkout
- Order confirmation
- Local Storage management

2.3 User Classes and Characteristics

2.3.1 Customer/User

The primary user who browses restaurants, searches for food, views prices and details, manages the cart, enters delivery information, and places an order. Assumed to have basic familiarity with web browsers and touchscreen/mouse-and-keyboard navigation. No account or technical expertise required.

2.3.2 Administrator

The current front-end version does not contain an administrator module. An admin panel — for managing restaurant/menu data without editing source files — is identified as a future enhancement (Section 16).

2.4 Operating Environment

- Client-side: any modern desktop or mobile web browser (Chrome, Edge, Firefox, Safari)
- No server-side runtime required for the current version
- Assets served either from the local filesystem or a lightweight static file/development server

2.5 Design and Implementation Constraints

- Must be implemented using HTML5, CSS3, and vanilla JavaScript (ES6), optionally styled with Bootstrap 5
- Must not depend on a backend database or server-side language
- Persistence, where implemented, is limited to what the browser's Local Storage API supports (~5–10 MB, string data only)
- All restaurant, menu, and pricing data is static/dummy data bundled with the application

2.6 Assumptions and Dependencies

- Users have a modern browser with JavaScript enabled
- Local Storage is available and not disabled/cleared by the user or browser privacy settings
- The device has a functioning internet connection to initially load the application and its static assets
- No real payment gateway, SMS, or email service is available or required in this version

3. Functional Requirements

Each functional requirement below is assigned an ID (for traceability, see Section 15), a priority, and a description. Priority levels: High (core to MVP), Medium (important but not blocking), Low (nice-to-have).

3.1 Home Page

ID	Requirement	Priority
FR-1.1	The system shall provide a home page containing a navigation bar with the application logo/name.	High
FR-1.2	The home page shall include a persistent search bar accessible from the navigation area.	High
FR-1.3	The home page shall include a hero section introducing the platform (banner image/text/CTA).	Medium
FR-1.4	The home page shall display a section of featured restaurants.	High
FR-1.5	The home page shall display a section of popular food items.	Medium
FR-1.6	The home page shall display selectable food category shortcuts (e.g., Pizza, Burgers, Desserts).	High
FR-1.7	The home page shall include a footer with basic site links/information.	Low

3.2 Restaurant Module

ID	Requirement	Priority
FR-2.1	The system shall display each restaurant using a card showing name, image, cuisine type, rating, and estimated delivery time.	High
FR-2.2	Each restaurant card shall show delivery information (e.g., delivery fee or free-delivery indicator).	Medium
FR-2.3	Each restaurant card shall provide a 'View Menu' action that navigates to that restaurant's menu.	High
FR-2.4	Restaurant listings shall support sorting (e.g., by rating or delivery time) where feasible.	Low

3.3 Food Menu Module

ID	Requirement	Priority
FR-3.1	The system shall display, for each food item: image, name, description, category, price, and rating (if available).	High
FR-3.2	Each food item shall have an 'Add to Cart' button.	High
FR-3.3	Food items shall be organized into categories such as Pizza, Burgers, Indian, Chinese, Desserts, and Beverages.	High
FR-3.4	The menu view shall indicate which restaurant the items belong to.	Medium

3.4 Search Module

ID	Requirement	Priority
FR-4.1	The system shall allow users to search for food items by name via a text input.	High
FR-4.2	The system shall compare the search input (case-insensitively) with available food names and display matching results.	High
FR-4.3	The system shall show an appropriate 'no results found' message when no item matches the search.	High
FR-4.4	Search results shall update dynamically as the user types (live search), where feasible.	Low

3.5 Filter Module

ID	Requirement	Priority
FR-5.1	The system shall allow users to filter food items by category.	High
FR-5.2	The system shall allow users to filter food items by price range.	Medium
FR-5.3	The system shall allow multiple filters to be applied simultaneously (e.g., category AND price range).	Medium
FR-5.4	The system shall allow users to clear/reset all active filters.	Medium

4. Shopping Cart Requirements

4.1 Add to Cart

The user shall be able to add a food item to the shopping cart. Detailed behavior:

- FR-6.1 (High): Clicking 'Add to Cart' increases the cart item count indicator in the navigation bar.
- FR-6.2 (High): The added item shall appear in the cart view with its name, image, unit price, and quantity.
- FR-6.3 (High): The initial quantity for a newly added item shall be one (1).
- FR-6.4 (Medium): Adding the same item again shall increment its existing quantity rather than creating a duplicate line entry.

4.2 Update Quantity

- FR-6.5 (High): The user shall be able to increase or decrease the quantity of an item using +/- controls.
- FR-6.6 (High): The system shall automatically recalculate the item's line total and the overall cart total when quantity changes.
- FR-6.7 (Medium): Quantity shall not be permitted to go below 1 via the decrease control (use Remove Item instead).

4.3 Remove Item

- FR-6.8 (High): The user shall be able to remove an item entirely from the cart.
- FR-6.9 (High): The cart count and total amount shall be recalculated immediately after removal.
- FR-6.10 (Medium): The system shall ask for confirmation before removing an item (optional UX safeguard).

4.4 Cart Storage

- FR-6.11 (Medium): The application may use browser Local Storage to save cart information so that it remains available after a page refresh.
- FR-6.12 (Low): The system shall gracefully handle the case where Local Storage is unavailable (e.g., private browsing) by falling back to in-memory state for the session.

4.5 Empty Cart State

- FR-6.13 (Medium): When the cart has no items, the system shall display an 'Your cart is empty' message with a call-to-action to browse restaurants.

5. Checkout Requirements

The checkout page shall display selected food items, quantities, prices, subtotal, delivery charge, final total, delivery address form, contact information, and payment method. Required delivery information must be validated before order confirmation.

5.1 Order Summary Display

- FR-7.1 (High): The checkout page shall list every cart item with quantity, unit price, and line total.
- FR-7.2 (High): The checkout page shall display subtotal, delivery charge, and final total.

5.2 Delivery Details Form

- FR-7.3 (High): The system shall provide input fields for customer name, contact number, and delivery address.
- FR-7.4 (High): The system shall validate that all required fields are non-empty before allowing checkout to proceed.
- FR-7.5 (Medium): The system shall validate the contact number format (numeric, appropriate length).
- FR-7.6 (Medium): The system shall display inline error messages next to invalid or missing fields.

5.3 Payment Method Selection

- FR-7.7 (High): The system shall allow the user to select a payment method (e.g., Cash on Delivery, Card — simulated, UPI — simulated).
- FR-7.8 (High): No real payment information shall be collected, transmitted, or stored (see Section 7.5 Security).

6. Order Requirements

After completing checkout, the user shall be able to confirm the order. The system shall validate required information, display an order confirmation and summary, show the total amount, and clear the cart after successful order placement. The current version simulates order placement rather than processing a real transaction.

- FR-8.1 (High): The system shall generate a simulated Order ID upon successful confirmation.
- FR-8.2 (High): The system shall display an order confirmation screen summarizing items, total amount, delivery address, and payment method.
- FR-8.3 (High): The system shall clear the cart automatically after a successful order placement.
- FR-8.4 (Medium): The system may store recent order details in Local Storage for reference within the session/browser.
- FR-8.5 (Low): The confirmation screen shall provide an option to return to the home page to place a new order.

7. Non-Functional Requirements

7.1 Usability

The application should provide simple navigation, clear buttons, readable text, intuitive layout, consistent design, and understandable messages. Interactive elements (buttons, links) shall have a minimum touch target size of ~44×44px on mobile layouts.

7.2 Performance

The application should load quickly, respond promptly to interactions, avoid unnecessary scripts, and efficiently update cart information. Target: initial page load under 3 seconds on a typical broadband connection; UI interactions (add to cart, filter, search) shall reflect changes in under 200ms.

7.3 Responsiveness

The application should work properly on desktop computers, laptops, tablets, and smartphones, using fluid layouts and breakpoints consistent with Bootstrap 5's grid system (e.g., ~576px, ~768px, ~992px, ~1200px).

7.4 Compatibility

The application should support modern browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari (current and one prior major version of each).

7.5 Security

No real payment information should be stored. Sensitive information should not be exposed, inputs should be validated (to prevent script injection in text fields), and real banking credentials should not be used or requested at any point.

7.6 Maintainability

Source code should be organized into HTML, CSS, and JavaScript files following separation of concerns. JavaScript functions should be clearly named, modular, and documented with comments where logic is non-trivial.

7.7 Reliability

The application shall behave predictably given the same input/state, and shall not lose cart data due to normal navigation (e.g., page refresh) when Local Storage persistence is implemented.

7.8 Scalability (Data)

The static data model (restaurants/menu items) shall be structured (e.g., as JSON) so that additional restaurants or menu items can be added without code changes to the rendering logic.

7.9 Accessibility

The UI shall aim to meet WCAG 2.1 Level AA where practical for a front-end-only project: sufficient color contrast, alt text on images, keyboard-navigable controls, and semantic HTML elements.

7.10 Portability

As a purely client-side application with no server dependency, FoodieHub shall run identically when opened locally, hosted on static hosting (e.g., GitHub Pages, Netlify), or served via a lightweight development server.

8. Hardware Requirements

The project does not require specialized hardware.

- Processor: Dual-Core processor or above
- RAM: 4 GB or above
- Storage: At least 500 MB free space
- Display: 1366 × 768 or higher
- Keyboard and mouse (or touchscreen for mobile devices)
- Internet connection for downloading dependencies/resources

9. Software Requirements

Category	Requirement
Operating System	Windows 10/11, Linux, or macOS
Front-End	HTML5, CSS3, JavaScript
Framework	Bootstrap 5 (Optional)
Code Editor	Visual Studio Code
Browser	Google Chrome / Edge / Firefox / Safari
Storage	Browser Local Storage
Version Control	Git/GitHub (Optional)

10. System Constraints

- No real backend server
- No centralized database
- Payment processing is simulated
- Restaurant information is static/dummy data
- No real-time order tracking
- No user authentication
- Orders are stored locally if Local Storage is used
- Application depends on the browser for data persistence

11. Data Requirements

11.1 Restaurant Data

Field	Type	Description
Restaurant ID	String/Number	Unique identifier for the restaurant
Restaurant Name	String	Display name
Restaurant Image	String (URL/path)	Path to restaurant image asset
Cuisine	String	Cuisine type, e.g., Italian, Chinese
Rating	Number (0-5)	Average customer rating
Delivery Time	String	Estimated delivery window, e.g., '25-35 min'
Menu Items	Array of Food ID	List of food items offered by the restaurant

11.2 Food Data

Field	Type	Description
Food ID	String/Number	Unique identifier for the food item
Food Name	String	Display name
Description	String	Short description of the dish
Category	String	e.g., Pizza, Burgers, Desserts
Price	Number	Unit price in local currency
Image	String (URL/path)	Path to food image asset
Restaurant ID	String/Number	Foreign key linking to the owning restaurant

11.3 Cart Data

Field	Type	Description
Food ID	String/Number	References the food item
Food Name	String	Denormalized name for quick display
Price	Number	Unit price at time of adding
Quantity	Number	Number of units in cart, minimum 1
Total Price	Number	Price × Quantity

11.4 Order Data

Field	Type	Description
Order ID	String/Number	Unique, system-generated identifier
Customer Name	String	Entered at checkout

Field	Type	Description
Contact Number	String	Entered at checkout, validated for format
Delivery Address	String	Entered at checkout
Ordered Items	Array of Cart Data	Snapshot of cart at time of order
Total Amount	Number	Final total including delivery charge
Payment Method	String	Selected method (simulated)
Order Status	String	e.g., 'Confirmed' (simulated, static)

11.5 Data Storage Notes

- Restaurant and food data are read-only, bundled as static JSON/JS objects at build/authoring time.
- Cart and Order data are the only mutable data sets, held in application state and optionally mirrored to Local Storage.
- All monetary values shall be stored as numbers (not strings) to support correct calculation.

12. External Interface Requirements

12.1 User Interface

The interface should contain a navigation menu, search bar, restaurant cards, food cards, category buttons, Add-to-Cart buttons, shopping cart, checkout form, and order confirmation section. The interface should use consistent colors, typography, spacing, and button design.

12.1.1 Key Screens

- Home / Landing page
- Restaurant listing page
- Restaurant menu page
- Search results view
- Cart / mini-cart view
- Checkout page
- Order confirmation page

12.2 Browser Interface

The application shall run through a standard web browser. Users should be able to access it by opening the main HTML file or through a local development server.

12.3 Hardware Interfaces

None required beyond a standard input device (keyboard, mouse, or touchscreen) and display.

12.4 Software Interfaces

The application interfaces only with the browser's built-in Web Storage API (Local Storage) for persistence. No third-party API integrations exist in the current scope.

12.5 Communication Interfaces

Standard HTTP/HTTPS is used only to initially load static assets (HTML, CSS, JS, images) from wherever the site is hosted.

13. Use Case Specifications

This section expands the original use-case summary table into fully detailed use case specifications for traceability and test design.

13.1 Use Case Summary Table

Actor	Use Case
User	Browse Restaurants
User	View Menu
User	Search Food
User	Filter Food
User	Add Food to Cart
User	Update Quantity
User	Remove Food
User	View Cart
User	Enter Delivery Details
User	Select Payment Method
User	Place Order
User	View Order Confirmation

13.2 Detailed Use Cases

UC-01: Browse Restaurants

Use Case ID	UC-01
Use Case Name	Browse Restaurants
Actor(s)	User
Preconditions	Application is loaded; restaurant data is available.
Main Flow	1) User lands on the home page. 2) System displays a list/grid of restaurant cards. 3) User scrolls or navigates through the listing.
Alternate Flow	If no restaurants are available, system displays an empty-state message.
Postconditions	User has viewed the available restaurants and may select one.
Priority	High

UC-02: View Menu

Use Case ID	UC-02
Use Case Name	View Menu

Actor(s)	User
Preconditions	User is on the restaurant listing page.
Main Flow	1) User clicks 'View Menu' on a restaurant card. 2) System navigates to that restaurant's menu page. 3) System displays all food items for that restaurant grouped by category.
Alternate Flow	If the restaurant has no menu items, system displays an appropriate message.
Postconditions	User can view, search, filter, or add menu items to the cart.
Priority	High

UC-03: Search Food

Use Case ID	UC-03
Use Case Name	Search Food
Actor(s)	User
Preconditions	User is on a page displaying food items.
Main Flow	1) User types a query into the search bar. 2) System compares the query with available food names. 3) System displays matching results.
Alternate Flow	If no items match, system displays a 'no results found' message.
Postconditions	Matching food items are displayed and can be added to the cart.
Priority	High

UC-04: Filter Food

Use Case ID	UC-04
Use Case Name	Filter Food
Actor(s)	User
Preconditions	User is viewing a list of food items.
Main Flow	1) User selects one or more filter options (category, price range). 2) System applies filters and updates the displayed list.
Alternate Flow	User clears filters to return to the full list.
Postconditions	Only items matching the selected filters are displayed.
Priority	Medium

UC-05: Add Food to Cart

Use Case ID	UC-05
Use Case Name	Add Food to Cart

Actor(s)	User
Preconditions	User is viewing a food item.
Main Flow	1) User clicks 'Add to Cart'. 2) System adds the item to the cart with quantity 1, or increments quantity if already present. 3) System updates the cart count indicator.
Alternate Flow	None.
Postconditions	The item is present in the cart with correct quantity and price.
Priority	High

UC-06: Update Quantity

Use Case ID	UC-06
Use Case Name	Update Quantity
Actor(s)	User
Preconditions	The item exists in the cart.
Main Flow	1) User clicks + or - next to a cart item. 2) System updates the item's quantity. 3) System recalculates line total and cart total.
Alternate Flow	If decreasing below 1, system either blocks the action or prompts to remove the item instead.
Postconditions	Cart reflects the updated quantity and totals.
Priority	High

UC-07: Remove Food

Use Case ID	UC-07
Use Case Name	Remove Food
Actor(s)	User
Preconditions	The item exists in the cart.
Main Flow	1) User clicks 'Remove' on a cart item. 2) System deletes the item from the cart. 3) System recalculates cart count and total.
Alternate Flow	System may prompt for confirmation before removing.
Postconditions	The item no longer appears in the cart.
Priority	High

UC-08: View Cart

Use Case ID	UC-08
Use Case Name	View Cart

Actor(s)	User
Preconditions	None (cart may be empty or populated).
Main Flow	1) User opens the cart view. 2) System displays all cart items, quantities, prices, and total; or an empty-cart message.
Alternate Flow	N/A
Postconditions	User can proceed to checkout or continue browsing.
Priority	High

UC-09: Enter Delivery Details

Use Case ID	UC-09
Use Case Name	Enter Delivery Details
Actor(s)	User
Preconditions	Cart has at least one item; user has reached checkout.
Main Flow	1) User fills in name, contact number, and delivery address. 2) System validates the fields on submit/blur.
Alternate Flow	If validation fails, system displays field-level error messages and blocks progression.
Postconditions	Valid delivery details are captured for the order.
Priority	High

UC-10: Select Payment Method

Use Case ID	UC-10
Use Case Name	Select Payment Method
Actor(s)	User
Preconditions	User is on the checkout page.
Main Flow	1) User selects a payment method from the available (simulated) options.
Alternate Flow	N/A
Postconditions	A payment method is associated with the pending order.
Priority	High

UC-11: Place Order

Use Case ID	UC-11
Use Case Name	Place Order
Actor(s)	User

Preconditions	Delivery details are valid; a payment method is selected; cart is non-empty.
Main Flow	1) User clicks 'Place Order' / 'Confirm Order'. 2) System validates all required data. 3) System generates a simulated Order ID and confirmation. 4) System clears the cart.
Alternate Flow	If validation fails, system returns the user to the incomplete field(s).
Postconditions	An order confirmation is displayed; the cart is empty.
Priority	High

UC-12: View Order Confirmation

Use Case ID	UC-12
Use Case Name	View Order Confirmation
Actor(s)	User
Preconditions	An order has just been successfully placed.
Main Flow	1) System displays the confirmation screen with order ID, items, total, delivery address, and payment method. 2) User may return to the home page.
Alternate Flow	N/A
Postconditions	User has a summary/record of their simulated order.
Priority	Medium

14. Data Flow Diagram (DFD) Relationship

14.1 Context (Level 0) Overview

At the context level, the single external entity 'User' interacts with the FoodieHub system, sending inputs (search terms, filter selections, cart actions, delivery/payment details) and receiving outputs (restaurant/menu displays, cart state, order confirmation).

14.2 Level 1 Process Decomposition

DFD Process	SRS Module
P1 – View Menu	Restaurant & Menu Module
P2 – Search & Filter	Search & Filter Module
P3 – Manage Cart	Shopping Cart Module
P4 – Checkout	Checkout Module
P5 – Order Confirmation	Order Module

14.3 Data Stores

- Restaurant Data – restaurant and menu information (static, read-only)
- Cart Storage – selected items and quantities (mutable, session or Local-Storage-backed)
- Order Storage – order details (write-once per order, simulated)

14.4 Data Flow Narrative

Restaurant Data flows into P1 (View Menu), which renders menu items for the user. User actions in P2 (Search & Filter) read from Restaurant Data and narrow what P1 displays. Add/Update/Remove actions flow into P3 (Manage Cart), which reads and writes Cart Storage. At checkout, P4 reads Cart Storage plus user-entered delivery/payment data, validates it, and — on confirmation — passes a finalized order into P5, which writes to Order Storage, displays confirmation to the user, and clears Cart Storage.

15. Requirements Traceability Matrix (Sample)

A traceability matrix maps requirements to their originating use case and verification method. A representative sample is shown below; the full matrix should be maintained alongside the test plan as the project develops.

Req ID	Related Use Case	Verification Method
FR-2.3	UC-02	UI test — click 'View Menu' navigates correctly
FR-4.2 / FR-4.3	UC-03	UI test — search with match and no-match queries
FR-5.1 / FR-5.2	UC-04	UI test — apply category and price filters
FR-6.1-FR-6.4	UC-05	UI test — add item, verify count/quantity
FR-6.5-FR-6.7	UC-06	UI test — increase/decrease quantity, verify totals
FR-6.8-FR-6.9	UC-07	UI test — remove item, verify totals
FR-7.3-FR-7.6	UC-09	UI test — submit form with missing/invalid fields
FR-8.1-FR-8.3	UC-11	UI test — place order, verify confirmation and cart clear

16. Testing Plan

16.1 Testing Approach

Given the front-end-only scope, testing shall focus on manual and/or scripted UI testing (e.g., browser dev tools, manual test cases, or a lightweight framework such as Jest for pure JavaScript logic like price calculations).

16.2 Test Levels

- Unit-level: pure JS functions such as cart total calculation, quantity update logic, and search/filter matching.
- Integration-level: interaction between modules, e.g., adding an item on the menu page updates the cart badge in the navigation bar.
- UI/System-level: full user flows from browsing to order confirmation, run manually across target browsers and screen sizes.

16.3 Sample Test Cases

Test ID	Scenario	Expected Result
TC-01	Search for an existing food item	Matching item(s) are displayed
TC-02	Search for a non-existent food item	'No results found' message is shown
TC-03	Add an item to an empty cart	Cart count becomes 1; item appears with quantity 1
TC-04	Increase quantity of a cart item	Line total and cart total update correctly
TC-05	Remove the only item in the cart	Cart shows empty-state message; total is 0
TC-06	Submit checkout form with empty required fields	Validation errors are shown; order is not placed
TC-07	Successfully place an order	Confirmation screen shows correct summary; cart is cleared
TC-08	Refresh page after adding items (Local Storage enabled)	Cart contents persist after refresh
TC-09	Load application on a mobile-width viewport	Layout reflows correctly with no horizontal scroll

16.4 Regression Considerations

Because there is no backend, regression risk concentrates in shared client-side state (cart) and Local Storage schema. Any change to the cart or order data shape should be re-tested against add/update/remove/checkout/confirm flows and against a pre-existing Local Storage payload from a prior version.

17. Risk Analysis

Risk	Likelihood	Impact	Mitigation
Local Storage disabled or full	Low	Medium	Fall back to in-memory cart for the session; show a non-blocking notice
Inconsistent behavior across browsers	Medium	Medium	Test on Chrome, Edge, Firefox, Safari before release; avoid non-standard APIs
Data/price mismatches due to manual JSON edits	Medium	Medium	Validate static data with a schema or lint script before deployment
Poor performance with large dummy datasets	Low	Low	Paginate or lazy-load restaurant/menu lists if the dataset grows
Users expecting real payment/tracking	Medium	Low	Clearly label checkout as simulated; set expectations in UI copy
Accessibility gaps (keyboard/screen reader)	Medium	Medium	Use semantic HTML, alt text, and test with keyboard-only navigation

18. Future Enhancements

- User registration and login
- Real-time restaurant data
- Backend server integration
- Database integration using MongoDB/MySQL
- Online payment gateway
- GPS-based restaurant selection
- Live order tracking
- Restaurant management dashboard
- Delivery partner module
- Customer reviews and ratings
- Discount coupons
- AI-powered food recommendations
- Push notifications
- Order history and reorder functionality
- Administrator module for managing restaurant/menu data without editing source files
- Automated unit/integration test suite (e.g., Jest, Cypress)

19. Acceptance Criteria

- Home page loads correctly
- Restaurants and food items are displayed properly
- Search functionality works correctly
- Food filtering works correctly
- Users can add food items to the cart
- Users can increase/decrease quantities
- Users can remove items from the cart
- Total price is calculated correctly
- Checkout form accepts required information and validates missing/invalid fields
- Order confirmation is displayed after checkout
- Website is responsive on different screen sizes
- Local Storage maintains cart information where implemented
- Application functions correctly on all supported browsers (Section 7.4)
- No real payment or banking data is ever requested or stored

20. Glossary

Term	Meaning
Cart Badge	The small numeric indicator on the cart icon showing item count
Empty State	The UI shown when a list (cart, search results) has no items to display
Line Total	Price × Quantity for a single cart item
Dummy Data	Static, hard-coded sample data used in place of a live database
Simulated Order	An order that is recorded and confirmed within the app but not sent to a real payment or logistics system

21. Conclusion

The FoodieHub – Food Delivery Web Application is a front-end web development project designed to provide users with a simple, attractive, and responsive food ordering experience. The project demonstrates important front-end concepts including HTML structure, CSS styling, responsive design, JavaScript interactivity, DOM manipulation, form validation, filtering, cart management, and Local Storage.

Although the current system is limited to front-end functionality, its modular design allows it to be extended into a complete full-stack food delivery platform by adding authentication, backend APIs, databases, payment gateways, and real-time order tracking. This expanded SRS — with detailed use cases, a data dictionary, a risk register, and a testing plan — is intended to support that growth path while giving present-day developers and evaluators an unambiguous specification to build and test against.

Project Name: FoodieHub

Project Type: Front-End Web Application

Domain: Food Delivery / E-Commerce

Primary Technologies: HTML5, CSS3, JavaScript

Storage: Browser Local Storage

Target Users: Customers / Food Ordering Users